

Architektura systemów komputerowych

Sprawozdanie z laboratorium

Wykonawca: Roch Rara
Numer albumu: 183938
Grupa: L8
Prowadzący: dr. inż. Dawid Warchoł

Rzeszów, 2026 r.

Spis treści

1	Instrukcja I	2
1.1	Zadanie 3	2
1.1.1	Podpunkt a)	2
1.1.2	Podpunkt b)	2
1.2	Zadanie 4	2
1.3	Zadanie 5	3
1.4	Zadanie 6	4
1.5	Zadanie 7	6
1.5.1	Podpunkt a)	6
1.5.2	Podpunkt b)	6
1.5.3	Podpunkt c)	7
1.5.4	Podpunkt d)	7
1.5.5	Podpunkt e)	8
1.6	Zadanie 8	9
1.6.1	Podpunkt a)	9
1.6.2	Podpunkt b)	10
2	Instrukcja II	11
2.1	Zadanie 1	11
2.2	Zadanie 2	11
2.2.1	Podpunkt a)	11
2.2.2	Podpunkt b)	12
2.2.3	Podpunkt c)	13
2.2.4	Podpunkt d)	15
2.2.5	Podpunkt e)	16
2.2.6	Podpunkt f)	17
2.3	Zadanie 3	19
2.3.1	Podpunkt a)	19
2.3.2	Podpunkt b)	19
2.4	Zadanie 4	20

1 Instrukcja I

1.1 Zadanie 3

1.1.1 Podpunkt a)

```
1 /* USER CODE BEGIN WHILE */
2 while (1)
3 {
4     HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
5     HAL_Delay(125);
6 /* USER CODE END WHILE */
7 }
```

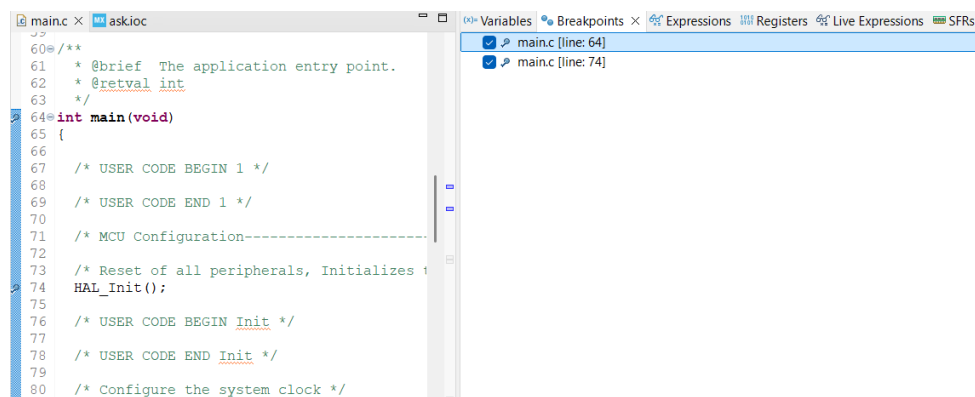
Listing 1: Kod dla podpunktu a.

1.1.2 Podpunkt b)

```
1 /* USER CODE BEGIN WHILE */
2 while (1)
3 {
4     HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
5     HAL_Delay(125);
6     HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
7     HAL_Delay(125);
8 /* USER CODE END WHILE */
9 }
```

Listing 2: Kod dla podpunktu b.

1.2 Zadanie 4



Rysunek 1: Zrzut ekranu do zadania 4.

1.3 Zadanie 5

```
1  /* USER CODE BEGIN PV */
2  GPIO_PinState stan_przycisku;
3  /* USER CODE END PV */
4  .....
5  /* USER CODE BEGIN WHILE */
6  while (1)
7  {
8      stan_przycisku = HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin);
9  /* USER CODE END WHILE */
10 }
```

Listing 3: Kod do zadania 5.

Expression	Type	Value
 stan_przycisku	GPIO_PinState	GPIO_PIN_SET

Rysunek 2: Zrzut ekranu do zadania kiedy przycisk puszczony 5.

Expression	Type	Value
 stan_przycisku	GPIO_PinState	GPIO_PIN_RESET

Rysunek 3: Zrzut ekranu do zadania kiedy przycisk kliknięty 5.

1.4 Zadanie 6

Name	Value
▼  General Registers	
 r0	1
 r1	8192
 r2	1
 r3	8192
 r4	536870960
 r5	4294967295
 r6	4294967295
 r7	536903672
 r8	4294967295
 r9	4294967295
 r10	4294967295
 r11	4294967295
 r12	4294967295
 sp	0x20007ff8
 lr	134218303
 pc	0x8000234 <main+20>
 xpsr	16777216
 msp	0x20007ff8
 psp	0xfffffff8
 primask	0
 basepri	0
 faultmask	0
 control	0

Rysunek 4: Zrzut ekranu do zadania 6.

Name	Value
General Registers	
 r0	0
 r1	8192
 r2	1342179328
 r3	8192
 r4	536870960
 r5	4294967295
 r6	4294967295
 r7	536903672
 r8	4294967295
 r9	4294967295
 r10	4294967295
 r11	4294967295
 r12	4294967295
 sp	0x20007ff8
 lr	134218303
 pc	0x8000238 <main+24>
 xpsr	16777216
 msp	0x20007ff8
 psp	0xffffffffc
 primask	0
 basepri	0
 faultmask	0
 control	0

Rysunek 5: Zrzut ekranu do zadania 6.

Odpowiedź

Rejestr r0, przechowujący wartość zwracaną przez funkcję, przyjmuje wartość 1 gdy przycisk jest puszczony (stan wysoki) oraz 0 gdy przycisk jest wciśnięty (stan niski). Rejestry r1 oraz r3 utrzymują stałą wartość 8192, co odpowiada masce bitowej pinu trzynastego (2^{13}).

W rejestrach ogólnych modyfikowanych jest wiele bitów jednocześnie, ponieważ przechowują one całe liczby (wartości zmiennych lub adresy pamięci), a nie pojedyncze linie sygnałowe.

1.5 Zadanie 7

1.5.1 Podpunkt a)

```
1  /* USER CODE BEGIN PV */
2  GPIO_PinState stan_przycisku;
3  /* USER CODE END PV */
4  .....
5  /* USER CODE BEGIN WHILE */
6  while (1)
7  {
8      stan_przycisku = HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin);
9      if (stan_przycisku == GPIO_PIN_RESET)
10     {
11         HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
12                             GPIO_PIN_SET);
13     }
14     else
15     {
16         HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
17                             GPIO_PIN_RESET);
18     }
19 /* USER CODE END WHILE */
20 }
```

Listing 4: Kod dla podpunktu a.

1.5.2 Podpunkt b)

```
1  /* USER CODE BEGIN PV */
2  GPIO_PinState stan_przycisku;
3  GPIO_PinState poprzedni_stan = GPIO_PIN_SET;
4  /* USER CODE END PV */
5  .....
6  /* USER CODE BEGIN WHILE */
7  while (1)
8  {
9      stan_przycisku = HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin);
10
11     if (poprzedni_stan == GPIO_PIN_RESET &&
12         stan_przycisku == GPIO_PIN_SET)
13     {
14         HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
15     }
16     poprzedni_stan = stan_przycisku;
17 /* USER CODE END WHILE */
18 }
```

Listing 5: Kod dla podpunktu b.

1.5.3 Podpunkt c)

```
1  /* USER CODE BEGIN PV */
2  GPIO_PinState stan_przycisku;
3  GPIO_PinState poprzedni_stan = GPIO_PIN_SET;
4  uint32_t opoznienia[] = {20, 50, 100, 500, 1000};
5  uint8_t indeks_czestotliwosci = 0;
6  void changeLEDFrequency(void);
7  /* USER CODE END PV */
8  .....
9  /* USER CODE BEGIN WHILE */
10 while (1)
11 {
12     HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
13     changeLEDFrequency();
14     HAL_Delay(opoznienia[indeks_czestotliwosci]);
15     /* USER CODE END WHILE */
16 }
17 .....
18 /* USER CODE BEGIN 4 */
19 void changeLEDFrequency(void)
20 {
21     stan_przycisku = HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin);
22
23     if (poprzedni_stan == GPIO_PIN_SET && stan_przycisku ==
24     GPIO_PIN_RESET)
25     {
26         indeks_czestotliwosci++;
27
28         if (indeks_czestotliwosci >= 5)
29         {
30             indeks_czestotliwosci = 0;
31         }
32     }
33     poprzedni_stan = stan_przycisku;
34 }
35 /* USER CODE END 4 */
```

Listing 6: Kod dla podpunktu c.

1.5.4 Podpunkt d)

```
1  /* USER CODE BEGIN PV */
2  GPIO_PinState stan_przycisku;
3  GPIO_PinState poprzedni_stan = GPIO_PIN_SET;
4  uint32_t opoznienia[] = {20, 50, 100, 500, 1000};
5  uint8_t indeks_czestotliwosci = 0;
6  uint32_t ostatni_czas_migania = 0;
7  void changeLEDFrequency(void);
8  /* USER CODE END PV */
9  .....
```

```

10  /* USER CODE BEGIN WHILE */
11  while (1)
12  {
13      changeLEDFrequency();
14      if (HAL_GetTick() - ostatni_czas_migania >= opoznienia[
indeks_czestotliwosci])
15      {
16          HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
17          ostatni_czas_migania = HAL_GetTick();
18      }
19      /* USER CODE END WHILE */
20  }
21  .....
22  /* USER CODE BEGIN 4 */
23  void changeLEDFrequency(void)
24  {
25      stan_przycisku = HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin);
26
27      if (poprzedni_stan == GPIO_PIN_SET && stan_przycisku ==
GPIO_PIN_RESET)
28      {
29          indeks_czestotliwosci++;
30
31          if (indeks_czestotliwosci >= 5)
32          {
33              indeks_czestotliwosci = 0;
34          }
35      }
36      poprzedni_stan = stan_przycisku;
37  }
38  /* USER CODE END 4 */

```

Listing 7: Kod dla podpunktu d.

Odpowiedź

Podczas wolniejszego migania funkcja HALDELAY wprowadza procesor w stan blokady. Mikrokontroler zostaje wówczas „zamrożony” na określony czas i fizycznie nie wykonuje kodu odpowiedzialnego za sprawdzenie stanu przycisku, co powoduje ignorowanie kliknięć.

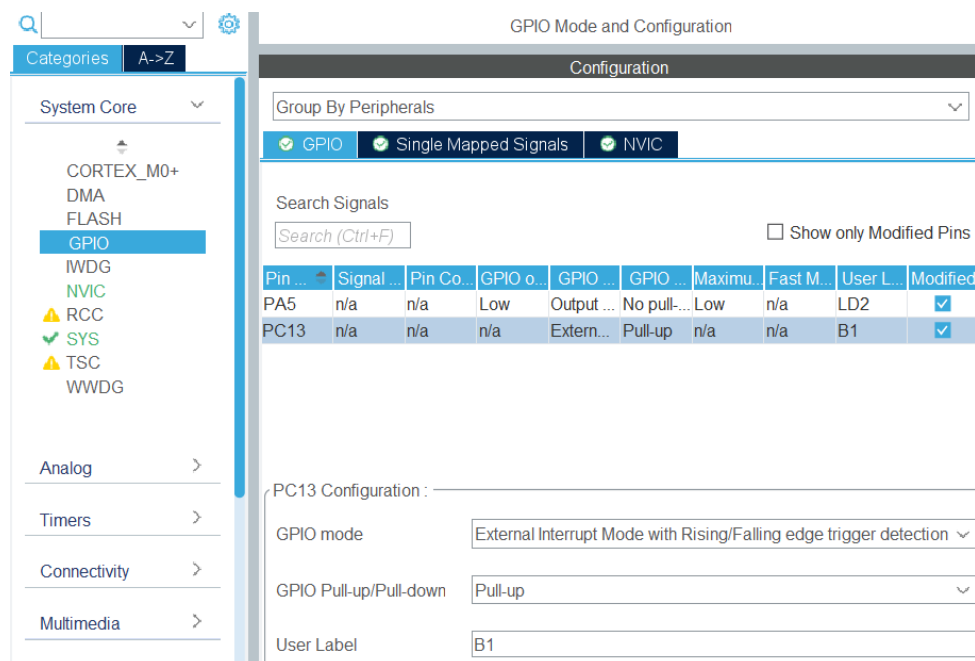
1.5.5 Podpunkt e)

Odpowiedź

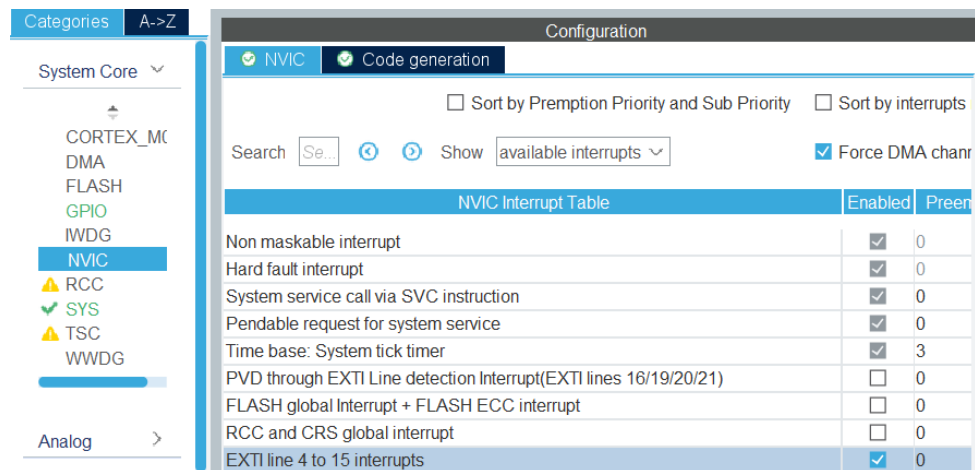
Zatrzymanie programu (stan resetu) następuje natychmiast po wciśnięciu czarnego przycisku – wtedy procesor jest zablokowany, a dioda przestaje migać. Ponowne uruchomienie programu (restart od początku, czyli od częstotliwości 25Hz) następuje dopiero po zwolnieniu (puszczeniu) przycisku RESET.

1.6 Zadanie 8

1.6.1 Podpunkt a)



Rysunek 6: Zrzut ekranu do zadania 8.



Rysunek 7: Zrzut ekranu do zadania 8.

```

1  /* USER CODE BEGIN PV */
2  GPIO_PinState stan_przycisku;
3  /* USER CODE END PV */
4  ....
5  /* USER CODE BEGIN 4 */
6  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
7  {
8      if (GPIO_Pin == B1_Pin)
9      {
10         stan_przycisku = HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin);
11
12         if (stan_przycisku == GPIO_PIN_RESET)
13         {
14             HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET)
15         ;
16         }
17         else
18         {
19             HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
20             GPIO_PIN_RESET);
21         }
22     }
23 }
24 /* USER CODE END 4 */

```

Listing 8: Kod do zadania 8 pudpunkt a.

1.6.2 Podpunkt b)

```

1  /* USER CODE BEGIN PV */
2  GPIO_PinState stan_przycisku;
3  GPIO_PinState poprzedni_stan_przycisku = GPIO_PIN_RESET;
4  /* USER CODE END PV */
5  ....
6  /* USER CODE BEGIN 4 */
7  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
8  {
9      if (GPIO_Pin == B1_Pin)
10     {
11         stan_przycisku = HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin);
12         if (stan_przycisku == GPIO_PIN_SET &&
13         poprzedni_stan_przycisku == GPIO_PIN_RESET)
14         {
15             HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
16         }
17         poprzedni_stan_przycisku = stan_przycisku;
18     }
19 }
20 /* USER CODE END 4 */

```

Listing 9: Kod do zadania 8 pudpunkt a.

2 Instrukcja II

2.1 Zadanie 1

LPUART1 Mode and Configuration

Mode

Mode

Configuration

Reset Configuration

✓ DMA Settings

✓ User Constants

✓ GPIO Settings

✓ NVIC Settings

✓ Parameter Settings

Configure the below parameters :

⏪

⏩

i

▼ Basic Parameters

Baud Rate

57600 Bits/s

Word Length

8 Bits (including Parity)

Parity

None

Stop Bits

1

▼ Advanced Parameters

Data Direction

Receive and Transmit

Single Sample

Disable

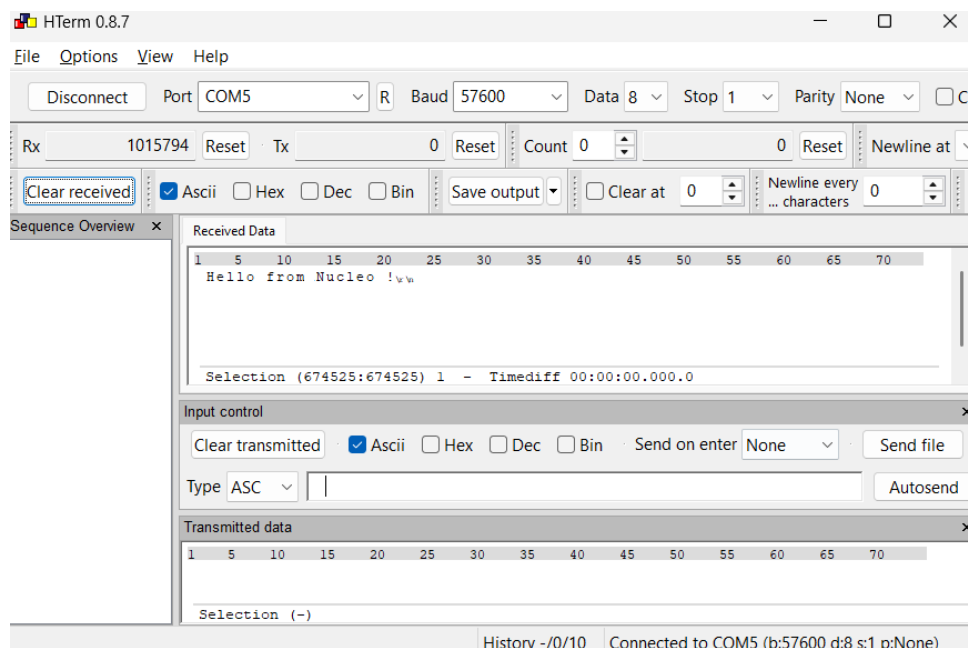
Rysunek 8: Zrzut ekranu do zadania 9.

2.2 Zadanie 2

2.2.1 Podpunkt a)

```
1 /* USER CODE BEGIN Includes */
2 #include <string.h>
3 /* USER CODE END Includes */
4 /* USER CODE BEGIN 2 */
5 char komunikat_powitalny[] = "Hello from Nucleo!\r\n";
6 HAL_UART_Transmit(&hlpuart1, (uint8_t*)komunikat_powitalny, strlen(
    komunikat_powitalny), 1000);
7 /* USER CODE END 2 */
```

Listing 10: Kod do zadania 8 pudpunkt a.



Rysunek 9: Zrzut ekranu do zadania 2 podpunkt a.

Odpowiedź

Określa maksymalny czas (w milisekundach), przez jaki funkcja HALUARTTransmit próbuje wysłać dane. Ponieważ to tryb blokujący, procesor czeka i nie robi nic innego. Jeśli w tym czasie transmisja się nie powiedzie, funkcja przerywa pracę i zwraca błąd, zapobiegając wiecznemu zawieszeniu programu.

Maksymalna wartość to HALMAXDELAY (czyli systemowo 0xFFFFFFFF). Przekazanie tej wartości całkowicie wyłącza odliczanie czasu – funkcja będzie czekać w nieskończoność, aż wszystkie bajty zostaną pomyślnie wysłane.

2.2.2 Podpunkt b)

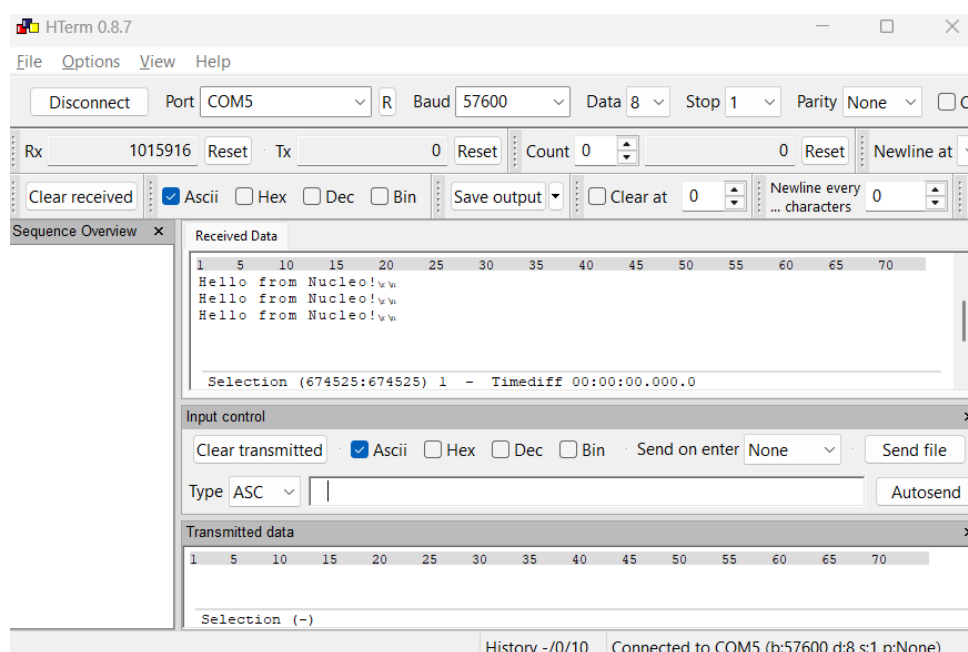
```
1 /* USER CODE BEGIN PV */
2 char komunikat_powitalny[] = "Hello from Nucleo!\r\n";
3 GPIO_PinState stan_przycisku;
4 GPIO_PinState poprzedni_stan = GPIO_PIN_SET;
5 /* USER CODE END PV */
6 /* USER CODE BEGIN WHILE */
7 while (1)
8 {
9     stan_przycisku = HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin);
10
11     if (stan_przycisku == GPIO_PIN_SET && poprzedni_stan ==
12         GPIO_PIN_RESET)
13     {
14         HAL_Delay(20);
```

```

14         if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) ==
15         GPIO_PIN_SET)
16         {
17             HAL_UART_Transmit(&hlpuart1, (uint8_t*)
18             komunikat_powitalny, strlen(komunikat_powitalny), HAL_MAX_DELAY)
19             ;
20         }
21     }
22     poprzedni_stan = stan_przycisku;
23 /* USER CODE END WHILE */
24 }

```

Listing 11: Kod do zadania 2 pudpunkt b.



Rysunek 10: Zrzut ekranu do zadania 2 podpunkt b.

2.2.3 Podpunkt c)

```

1  /* USER CODE BEGIN PV */
2  GPIO_PinState stan_przycisku;
3  GPIO_PinState poprzedni_stan = GPIO_PIN_SET;
4  uint32_t czas_wcisniecia = 0;
5  uint32_t czas_trzymania = 0;
6  /* USER CODE END PV */
7  .....
8  /* USER CODE BEGIN WHILE */
9  while (1)
10  {
11      stan_przycisku = HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin);
12

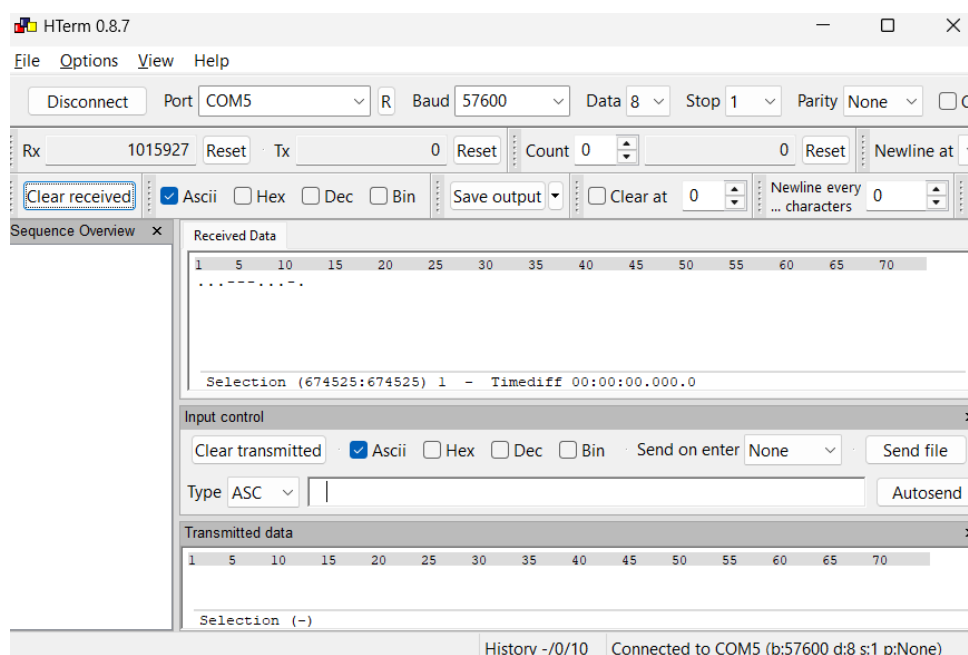
```

```

13     if (stan_przycisku == GPIO_PIN_RESET && poprzedni_stan ==
GPIO_PIN_SET)
14     {
15         HAL_Delay(20);
16         czas_wcisniecia = HAL_GetTick();
17     }
18
19     if (stan_przycisku == GPIO_PIN_SET && poprzedni_stan ==
GPIO_PIN_RESET)
20     {
21         HAL_Delay(20);
22         czas_trzymania = HAL_GetTick() - czas_wcisniecia;
23
24         if (czas_trzymania >= 500)
25         {
26             HAL_UART_Transmit(&hlpuart1, (uint8_t*)"-", 1,
HAL_MAX_DELAY);
27         }
28         else if (czas_trzymania > 50)
29         {
30             HAL_UART_Transmit(&hlpuart1, (uint8_t*)".", 1,
HAL_MAX_DELAY);
31         }
32     }
33     poprzedni_stan = stan_przycisku;
34     /* USER CODE END WHILE */
35 }
36 }

```

Listing 12: Kod do zadania 2 pudpunkt c.

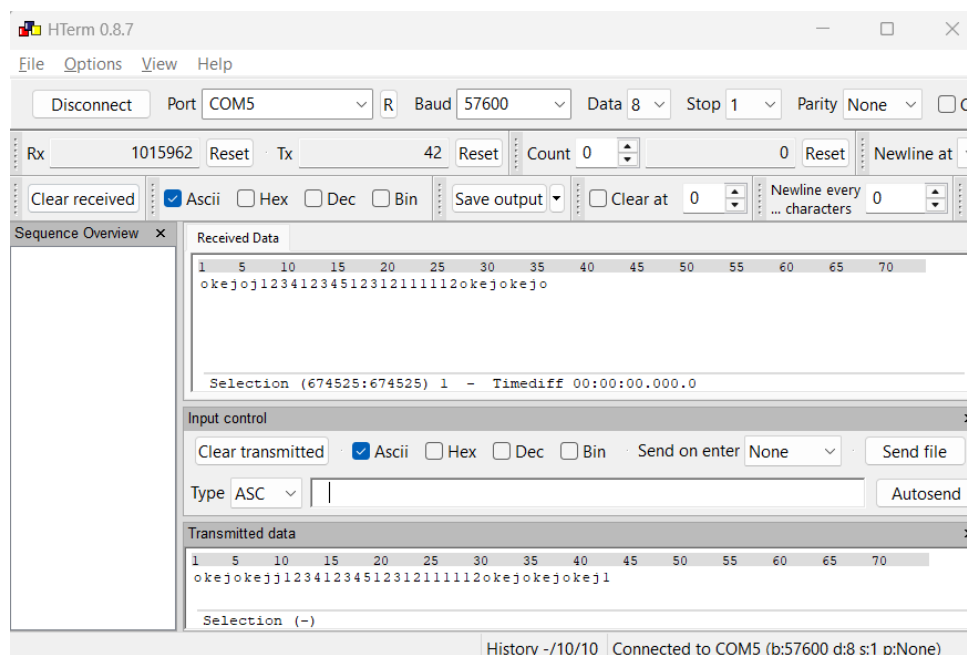


Rysunek 11: Zrzut ekranu do zadania 2 podpunkt c.

2.2.4 Podpunkt d)

```
1  /* USER CODE BEGIN PV */
2  uint8_t bufor_odbiorczy[5];
3  /* USER CODE END PV */
4  .....
5  /* USER CODE BEGIN WHILE */
6  while (1)
7  {
8      HAL_UART_Receive(&hlpuart1, bufor_odbiorczy, 5, HAL_MAX_DELAY)
9      ;
10     HAL_UART_Transmit(&hlpuart1, bufor_odbiorczy, 5, HAL_MAX_DELAY
11 );
12 /* USER CODE END WHILE */
13 }
```

Listing 13: Kod do zadania 2 pudpunkt d.



Rysunek 12: Zrzut ekranu do zadania 2 podpunkt d.

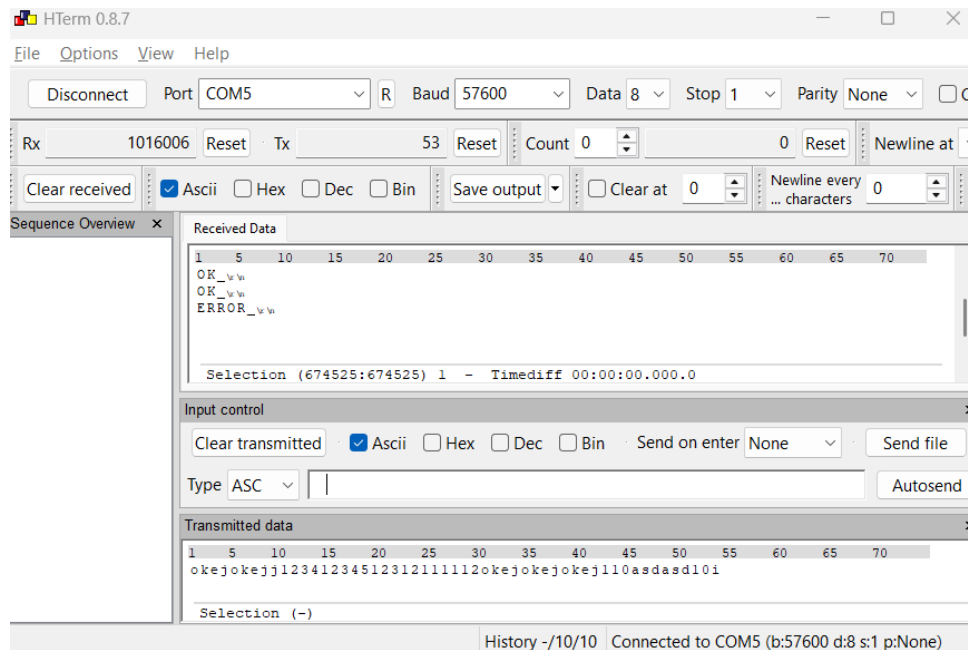
Odpowiedź

Przy wysłaniu komunikatu krótszego niż 5 znaków, program zawiesi się na instrukcji odbioru i nie odeśle danych, dopóki bufor nie zostanie dopełniony brakującymi literami. Jeśli natomiast wyślemy zbyt długi tekst, procesor przechwyci i odeśle tylko pierwszych 5 znaków. Reszta nadmiarowych danych nie przepadnie — zostanie w pamięci podręcznej portu szeregowego i wywoła natychmiastową reakcję programu w następnym cyklu pętli while(1).

2.2.5 Podpunkt e)

```
1  /* USER CODE BEGIN PV */
2  uint8_t odebrany_znak;
3  /* USER CODE END PV */
4  /* USER CODE BEGIN WHILE */
5      while (1)
6      {
7          HAL_UART_Receive(&hlpuart1, &odebrany_znak, 1,
8          HAL_MAX_DELAY);
9
10         if (odebrany_znak == '1')
11         {
12             HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET)
13             ;
14
15             HAL_UART_Transmit(&hlpuart1, (uint8_t*)"OK_\r\n", 5,
16             HAL_MAX_DELAY);
17         }
18         else if (odebrany_znak == '0')
19         {
20             HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
21             GPIO_PIN_RESET);
22
23             HAL_UART_Transmit(&hlpuart1, (uint8_t*)"OK_\r\n", 5,
24             HAL_MAX_DELAY);
25         }
26         else
27         {
28             HAL_UART_Transmit(&hlpuart1, (uint8_t*)"ERROR_\r\n", 8,
29             HAL_MAX_DELAY);
30         }
31     }
32 /* USER CODE END WHILE */
33 }
```

Listing 14: Kod do zadania 2 pudpunkt e.



Rysunek 13: Zrzut ekranu do zadania 2 podpunkt e.

2.2.6 Podpunkt f)

```

1  /* USER CODE BEGIN PV */
2  uint8_t odebrany_znak;
3  /* USER CODE END PV */
4  /* USER CODE BEGIN WHILE */
5  while (1)
6  {
7      if (HAL_UART_Receive(&hlpuart1, &odebrany_znak, 1,
8          HAL_MAX_DELAY) == HAL_OK)
9      {
10         if (odebrany_znak >= '0' && odebrany_znak <= '9')
11         {
12             int liczba_migniec = odebrany_znak - '0';
13             for (int i = 0; i < liczba_migniec; i++)
14             {
15                 HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
16                     GPIO_PIN_SET);
17                 HAL_Delay(250);
18                 HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
19                     GPIO_PIN_RESET);
20                 HAL_Delay(250);
21             }
22         }
23         else
24         {
25             HAL_UART_Transmit(&hlpuart1, (uint8_t*)"ERROR\r\n",
26                 7, HAL_MAX_DELAY);

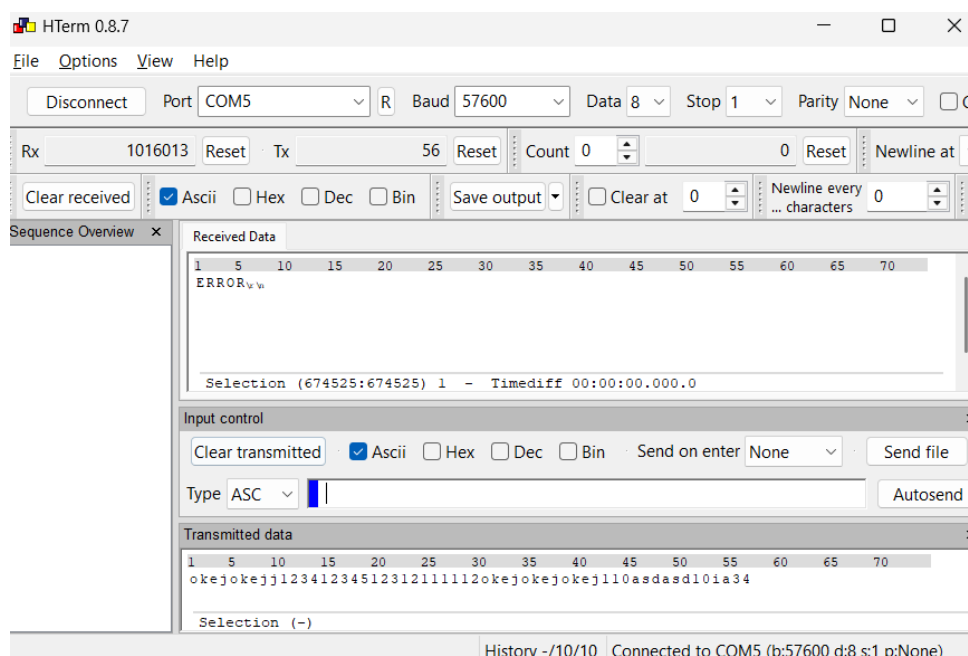
```

```

25     }
26 }
27 /* USER CODE END WHILE */
28 }

```

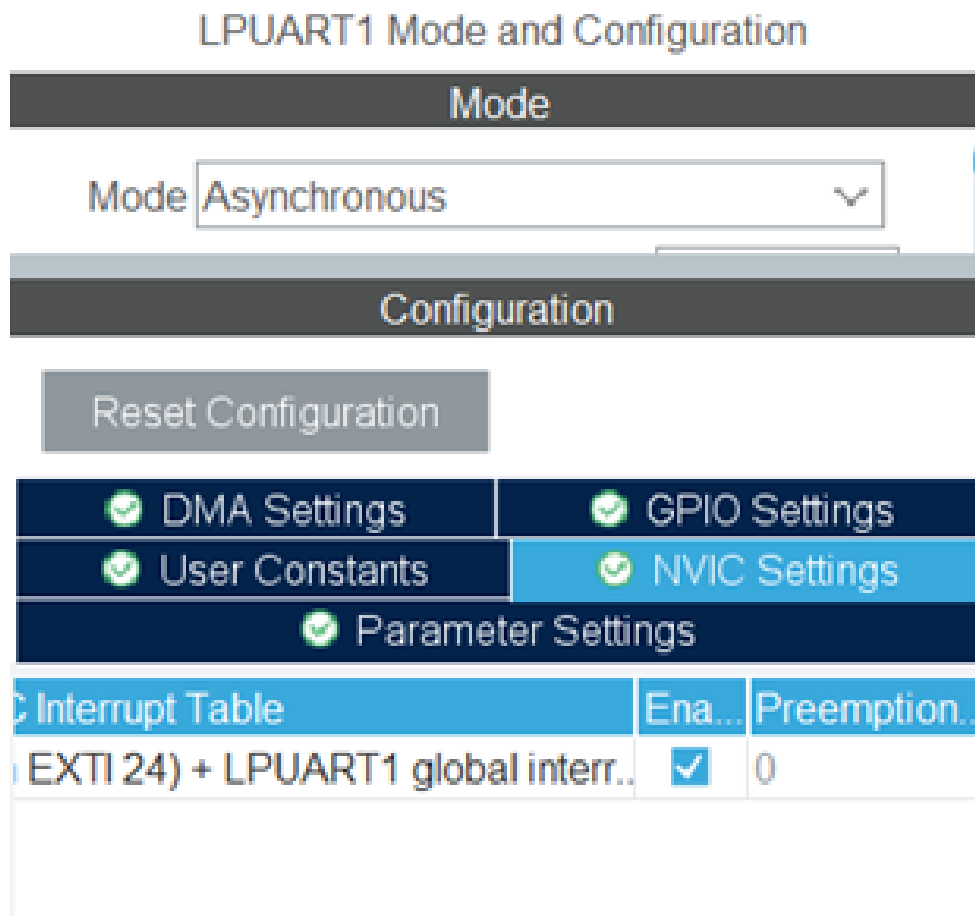
Listing 15: Kod do zadania 2 pudpunkt f.



Rysunek 14: Zrzut ekranu do zadania 2 podpunkt f.

2.3 Zadanie 3

2.3.1 Podpunkt a)



Rysunek 15: Zrzut ekranu do zadania 3 podpunkt a.

2.3.2 Podpunkt b)

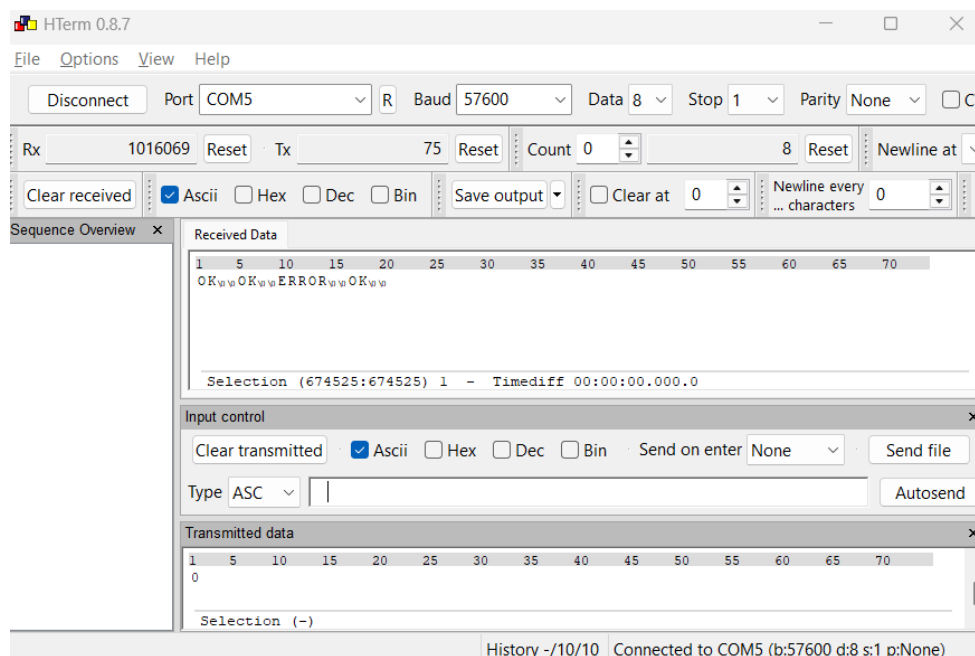
```
1  /* USER CODE BEGIN PV */
2  uint8_t odebrany_znak;
3  /* USER CODE END PV */
4  .....
5  /* USER CODE BEGIN 4 */
6  void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
7  {
8      if (huart->Instance == LPUART1)
9      {
10         if (odebrany_znak == '1')
11         {
12             HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET)
13             ;
14             HAL_UART_Transmit_IT(&hlpuart1, (uint8_t*)"OK", 4);
15         }
16         else if (odebrany_znak == '0')
```

```

16     {
17         HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
18         GPIO_PIN_RESET);
19         HAL_UART_Transmit_IT(&hlpuart1, (uint8_t*)"OK", 4);
20     }
21     else
22     {
23         HAL_UART_Transmit_IT(&hlpuart1, (uint8_t*)"ERROR", 7);
24     }
25     HAL_UART_Receive_IT(&hlpuart1, &odebrany_znak, 1);
26 }
27 /* USER CODE END 4 */

```

Listing 16: Kod do zadania 3 podpunkt b.



Rysunek 16: Zrzut ekranu do zadania 3 podpunkt b.

2.4 Zadanie 4

```

1  /* USER CODE BEGIN PV */
2  uint8_t bufor_morse_odbior[100];
3  /* USER CODE END PV */
4  .....
5  /* USER CODE BEGIN 2 */
6  HAL_UARTEx_ReceiveToIdle_IT(&hlpuart1, bufor_morse_odbior, 100);
7  /* USER CODE END 2 */
8  .....
9  /* USER CODE BEGIN 4 */
10 void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t
    Size)
11 {

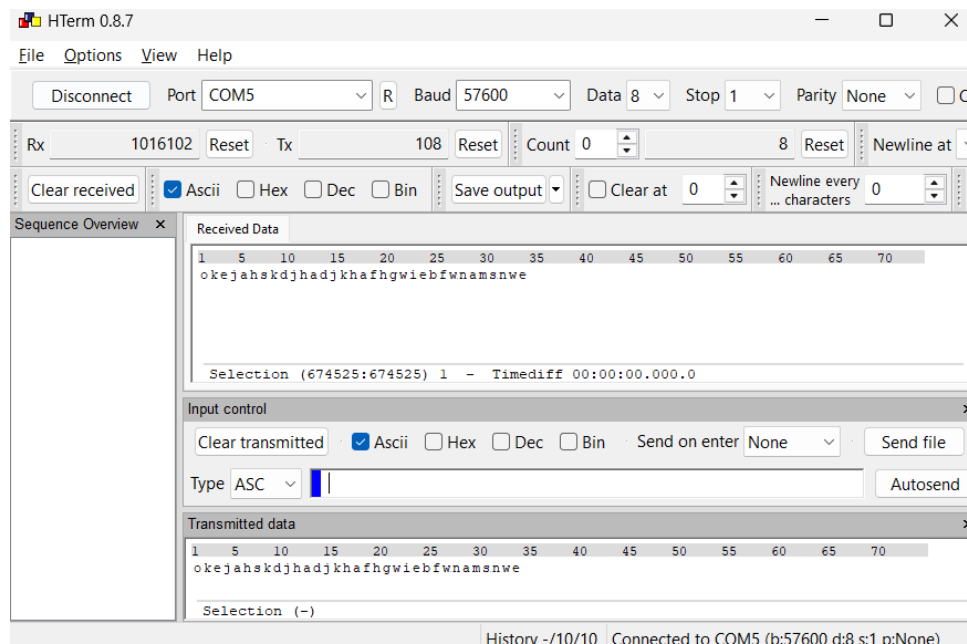
```

```

12  if (huart->Instance == LPUART1)
13  {
14      HAL_UART_Transmit_IT(&hlpuart1, bufor_morse_odbior, Size);
15      HAL_UARTEx_ReceiveToIdle_IT(&hlpuart1, bufor_morse_odbior,
16      100);
17  }
18  }
19  /* USER CODE END 4 */

```

Listing 17: Kod do zadania 4.



Rysunek 17: Zrzut ekranu do zadania 4.

Odpowiedź

Pierwszym przypadkiem jest kiedy nastąpi całkowite zapełnienie bufora, po odebraniu setnego bajta danych. Natomiast drugą sytuacją jest wykrycie bezczynności IDLE, tzn. że komputer nie przesyłał żadnych danych przez czas przynajmniej jednej ramki.